Lvalue and Rvalue References

Theory, Edge Cases & Code Examples

- ✓ Understanding Value Categories
- ✓ Lvalue References vs Rvalue References
- ✓ The Critical Distinction
- ✓ Named Rvalue References Are Lvalues
- ✓ Binding Rules for References

→ Swipe to learn →

Lvalue and Rvalue References

Theory

📌 1. Lvalues vs Rvalues - Understanding Value Categories (1/3)

In C++, every expression has two fundamental, **independent** properties: a **type** and a **value category**. Mastering value categories is essential for understanding move semantics, perfect forwarding, and modern C++ optimization.

Lvalue vs Rvalue Fundamental Differences

Aspect	Lvalue (Locator Value)	Rvalue (Read Value)
Memory Location	Has identifiable, persistent memory location	Temporary, no stable addressable location
Addressability	Can take address with `&` operator	Cannot take address (no stable location)
Lifetime	Persists beyond single expression	Exists only within expression
Examples	Variables, array elements, `*ptr`, lvalue refs	Literals, temporaries, expression results, `x+y`
Historical Name Origin	Can appear on **left** side of assignment	Originally "right side values" (read-only)
Modifiability	Can be modified (if non-const)	Can be modified through rvalue ref binding
Reference Binding	Binds to `T&` and `const T&`	Binds to `T&&` and `const T&`
Usage Context	When you need persistent object	When using temporary/expiring object

Lvalue and Rvalue References

Theory

📌 1. Lvalues vs Rvalues - Understanding Value Categories (2/3)

Detailed Value Category Classification

Expression	Value Category	Has Address?	Can Modify?	Example
Variable `x`	lvalue	✅ Yes	✅ Yes (if non-const)	`int x = 10;`
Array element `arr[i]`	lvalue	✅ Yes	✅ Yes (if non-const)	`arr[5] = 42;`
Dereferenced pointer `*ptr`	lvalue	✅ Yes	✅ Yes (if non-const)	`*ptr = 100;`
Function returning `T&`	lvalue	✅ Yes	✅ Yes	`getRef()`
Literal `42`	prvalue (pure rvalue)	❌ No	❌ No	`42`
Expression `x + y`	prvalue	❌ No	❌ No	`5 + 10`
Temporary object	prvalue	❌ No	✅ Yes (through rvalue ref)	`std::string("temp")`
`std::move(x)`	xvalue (expiring value)	❌ No	✅ Yes (through rvalue ref)	`std::move(obj)`
Function returning `T&&`	xvalue	❌ No	✅ Yes	`getRvalueRef()`
String literal `"hello"`	lvalue (special case)	✅ Yes	❌ No (const char array)	`"hello"`

Lvalue and Rvalue References

Theory

📌 1. Lvalues vs Rvalues - Understanding Value Categories (3/3)

Code Example: Identifying Value Categories

```
int x = 10;
int* ptr = &x;

// ✅ LVALUES (have addresses, persist)
int& lref1 = x;             // Variable is lvalue
int& lref2 = *ptr;          // Dereferenced pointer is lvalue
int& lref3 = ++x;           // Pre-increment returns lvalue
auto p1 = &x;               // Can take address

// ✅ RVALUES (temporaries, no stable address)
int&& rref1 = 42;           // Literal is rvalue
int&& rref2 = x + 5;        // Expression result is rvalue
int&& rref3 = x++;          // Post-increment returns rvalue (copy of old value)
int&& rref4 = std::move(x); // std::move casts to rvalue

// ❌ ILLEGAL OPERATIONS
// int& bad1 = 42;          // Cannot bind non-const lvalue ref to rvalue
// int&& bad2 = x;          // Cannot bind rvalue ref to lvalue (need std::move)
// auto bad3 = &42;         // Cannot take address of rvalue
```

Lvalue and Rvalue References

Theory

📌 2. Lvalue References vs Rvalue References (1/4)

C++11 introduced rvalue references to enable move semantics and perfect forwarding, complementing traditional lvalue references.

Reference Types Comparison

Feature	Lvalue Reference (`T&`)	Rvalue Reference (`T&&`)	const Lvalue Reference (`const T&`)
Syntax	`int& ref`	`int&& rref`	`const int& cref`
Binds To	Lvalues only	Rvalues only	Both lvalues and rvalues
Primary Purpose	Aliasing, avoiding copies	Move semantics, resource transfer	Universal binding, read-only access
Can Modify?	✅ Yes (if `T` non-const)	✅ Yes	❌ No (const)
Introduced	C++98	C++11	C++98
Common Use	Function parameters, return values	Move constructors/assignment	Temporary binding, const-correctness
Example	`void f(int& x)`	`void f(int&& x)`	`void f(const int& x)`

Lvalue and Rvalue References

Theory

📌 2. Lvalue References vs Rvalue References (2/4)

Reference Binding Rules

Source Expression	Can Bind to `T&`?	Can Bind to `const T&`?	Can Bind to `T&&`?
Non-const lvalue (`int x`)	✅ Yes	✅ Yes	❌ No
Const lvalue (`const int cx`)	❌ No	✅ Yes	❌ No
Rvalue literal (`42`)	❌ No	✅ Yes (+ lifetime ext)	✅ Yes (+ lifetime ext)
Rvalue expression (`x + 5`)	❌ No	✅ Yes (+ lifetime ext)	✅ Yes (+ lifetime ext)
`std::move(x)`	❌ No	✅ Yes	✅ Yes
`std::move(const_x)`	❌ No	✅ Yes	❌ No (becomes `const T&&`)

Lvalue and Rvalue References

Theory

📌 2. Lvalue References vs Rvalue References (3/4)

Code Example: Reference Binding Behavior

```
void take_lvalue(int& x) { std::cout << "lvalue ref\n"; }
void take_const_lvalue(const int& x) { std::cout << "const lvalue ref\n"; }
void take_rvalue(int&& x) { std::cout << "rvalue ref\n"; }

int main() {
    int a = 10;
    const int ca = 20;

    // ✅ Lvalue reference binding
    take_lvalue(a);       // OK: a is lvalue
    // take_lvalue(10);   // ❌ Error: cannot bind rvalue to non-const lvalue ref
    // take_lvalue(ca);   // ❌ Error: cannot bind const lvalue to non-const ref

    // ✅ Rvalue reference binding
    take_rvalue(42);      // OK: literal is rvalue
    take_rvalue(a + 5);   // OK: expression result is rvalue
    // take_rvalue(a);    // ❌ Error: cannot bind lvalue to rvalue ref
    take_rvalue(std::move(a));  // ✅ OK: std::move casts lvalue to rvalue

    // ✅ const lvalue reference (universal acceptor)
    take_const_lvalue(a);        // OK: lvalue binds
    take_const_lvalue(ca);       // OK: const lvalue binds
    take_const_lvalue(30);       // OK: rvalue binds (lifetime extended!)
    take_const_lvalue(std::move(a)); // OK: rvalue binds
}
```

Lvalue and Rvalue References

Theory

📌 2. Lvalue References vs Rvalue References (4/4)

Why Rvalue References Were Introduced (C++11)

Problem Before C++11	Solution with Rvalue References
No way to detect temporaries	Overload resolution distinguishes `T&` vs `T&&`
Expensive copies from temporaries	Move constructors steal resources instead of copying
Inefficient returns from functions	Move semantics + RVO eliminate copies
No perfect forwarding	Universal references + `std::forward` preserve value category
Copying even when object is dying	Can safely "steal" from expiring temporaries

---

Lvalue and Rvalue References

Theory

📌 3. Type vs Value Category - The Critical Distinction (1/4)

The Most Confusing Aspect of C++: Type and value category are **independent properties**. A variable can have rvalue reference type (`T&&`) but be an lvalue expression!

Type vs Value Category Matrix

Declaration	Variable Type (`decltype`)	Expression Value Category	Can Pass to `void f(T&)`?	Can Pass to `void f(T&&)`?
`int x;`	`int`	lvalue	✅ Yes	❌ No
`int& ref = x;`	`int&`	lvalue	✅ Yes	❌ No
`int&& rref = 42;`	`int&&`	**lvalue** (named!)	✅ Yes	❌ No (need `std::move`)
Expression `42`	`int` (prvalue)	rvalue	❌ No	✅ Yes
Expression `std::move(x)`	`int&&` (xvalue)	rvalue	❌ No	✅ Yes

The Golden Rule: Named Entities Are Always Lvalues

Regardless of type, **if it has a name, it's an lvalue**. This includes:
- Variables declared with rvalue reference type: `int&& rref = 42;` → `rref` is lvalue
- Function parameters: `void f(int&& param)` → `param` is lvalue inside function
- Any named variable, even if bound to a temporary

Lvalue and Rvalue References

Theory

📌 3. Type vs Value Category - The Critical Distinction (2/4)

Code Example: Type vs Value Category

```
void process_rvalue(int&& x) {
    // IMPORTANT: x has TYPE int&&, but EXPRESSION x is lvalue!

    std::cout << "x has type int&&\n";
    std::cout << "But expression x is lvalue\n";

    int& lref = x;  // ✅ OK: x is lvalue, can bind lvalue ref
    // int&& rref = x;  // ❌ Error: cannot bind rvalue ref to lvalue

    // To pass x as rvalue, must explicitly cast:
    int&& rref = std::move(x);  // ✅ OK: std::move casts to rvalue
}

int main() {
    int&& myref = 42;  // myref TYPE: int&&, myref EXPRESSION: lvalue

    // myref is a named variable → it's an lvalue!
    int& alias = myref;  // ✅ OK: myref is lvalue

    // To pass myref to rvalue reference parameter:
    // process_rvalue(myref);  // ❌ Error: myref is lvalue
    process_rvalue(std::move(myref));  // ✅ OK: cast to rvalue
}
```

Lvalue and Rvalue References

Theory

📌 3. Type vs Value Category - The Critical Distinction (3/4)

Why This Matters for Overload Resolution

Function Call	Actual Argument	Argument Value Category	Selected Overload
`void f(T&); void f(T&&);`	`int x; f(x);`	lvalue	`f(T&)`
`void f(T&); void f(T&&);`	`f(42);`	rvalue	`f(T&&)`
`void f(T&); void f(T&&);`	`int&& r = 42; f(r);`	lvalue (named!)	`f(T&)`
`void f(T&); void f(T&&);`	`f(std::move(x));`	rvalue (xvalue)	`f(T&&)`
`void f(const T&); void f(T&&);`	`f(42);`	rvalue	`f(T&&)` (exact match)
Only `void f(const T&);`	`f(42);`	rvalue	`f(const T&)` (fallback)

Decision Tree: Do I Have an Lvalue or Rvalue?

```
Does the expression have a NAME?
├─ YES → It's an LVALUE
│   └─ Examples: variables, function parameters, array elements
└─ NO → Check further:
    ├─ Is it a function call?
    │   ├─ Returns T& → LVALUE
    │   ├─ Returns T&& → XVALUE (rvalue)
    │   └─ Returns T → PRVALUE (rvalue)
    ├─ Is it std::move(x)? → XVALUE (rvalue)
    ├─ Is it a literal (42, "hello")? → PRVALUE (rvalue)
    └─ Is it an expression (x+y, x++)? → PRVALUE (rvalue)
```

📌 3. Type vs Value Category - The Critical Distinction (4/4)

Key Takeaways:

1. **Type tells you what it is declared as** → `decltype(x)` gives `int&&`
2. **Value category tells you how you can use it** → expression `x` is lvalue
3. **Named rvalue references are lvalues** → most confusing C++ rule!
4. **Use `std::move` to cast lvalue to rvalue** → enables passing to rvalue ref parameters
5. **Overload resolution uses value category, not type** → determines which function is called

---

Lvalue and Rvalue References

Edge Cases

📌 1. Named Rvalue References Are Lvalues

One of the most counterintuitive aspects of C++ is that a variable of rvalue reference type is actually an lvalue. This seems contradictory but follows from the fundamental rule: any named entity is an lvalue.

```
void process(int&& x) {
    // Inside this function, x has TYPE int&& (rvalue reference)
    // But the EXPRESSION x is an lvalue (it has a name)

    int&& another = x;  // ❌ Error: cannot bind rvalue ref to lvalue
    int&& another = std::move(x);  // ✅ OK: std::move casts x back to rvalue
    int& lref = x;  // ✅ OK: x is an lvalue, can bind to lvalue ref
}

int main() {
    int&& rref = 42;  // rref has type int&&

    // But when you USE rref, it's an lvalue:
    process(rref);  // ❌ Error: passing lvalue to function expecting rvalue
    process(std::move(rref));  // ✅ OK: std::move casts to rvalue
}
```

This example demonstrates the critical distinction between type and value category. The variable `rref` is declared with type `int&&`, but any expression using `rref` is an lvalue because it's a named variable. To pass it to a function expecting an rvalue reference, you must explicitly cast it with `std::move`.

Lvalue and Rvalue References

Edge Cases

📌 2. Binding Rules for References

C++ has strict rules about what can bind to different reference types, and violating these rules results in compilation errors.

```
void take_lvalue(int& x) { }
void take_rvalue(int&& x) { }
void take_const_lvalue(const int& x) { }

int main() {
    int a = 10;

    // Lvalue reference binding:
    take_lvalue(a);      // ✅ OK: lvalue binds to lvalue ref
    take_lvalue(10);     // ❌ Error: cannot bind rvalue to non-const lvalue ref

    // Rvalue reference binding:
    take_rvalue(10);     // ✅ OK: rvalue binds to rvalue ref
    take_rvalue(a);      // ❌ Error: cannot bind lvalue to rvalue ref
    take_rvalue(std::move(a));  // ✅ OK: std::move casts lvalue to rvalue

    // Const lvalue reference (the universal binder):
    take_const_lvalue(a);   // ✅ OK: lvalue binds to const lvalue ref
    take_const_lvalue(10);  // ✅ OK: rvalue ALSO binds to const lvalue ref
}
```

The `const T&` is special because it can bind to both lvalues and rvalues, making it the "universal reference" of pre-C++11 code. However, it doesn't allow distinguishing between temporary and non-temporary objects, which is why rvalue references were introduced.

Lvalue and Rvalue References

Edge Cases

📌 3. Const and Rvalue References

The interaction between `const` and rvalue references is subtle and important for understanding why certain operations fail.

```
void modify(int&& x) {
    x = 100;  // ✅ OK: rvalue references are modifiable
}

void modify_const(const int&& x) {
    x = 100;  // ❌ Error: cannot modify const
}

int main() {
    const int cx = 5;

    // std::move(cx) produces const int&&
    int&& r1 = std::move(cx);  // ❌ Error: cannot bind int&& to const int&&
    const int&& r2 = std::move(cx);  // ✅ OK: types match

    modify(std::move(cx));  // ❌ Error: cannot pass const int&& to int&&
}
```

When you `std::move` a const object, you get a `const T&&`, which cannot bind to a non-const rvalue reference `T&&`. This is important because move operations typically need to modify the source object (e.g., setting pointers to nullptr), which is impossible with const objects.

Lvalue and Rvalue References

Edge Cases

📌 4. Returning References to Local Variables

Returning references (both lvalue and rvalue) to local variables is undefined behavior because the local variable is destroyed when the function returns.

```
int& bad_return_lvalue() {
    int x = 42;
    return x;  // ❌ UB: returning reference to local variable
}

int&& bad_return_rvalue() {
    int x = 42;
    return std::move(x);  // ❌ UB: std::move doesn't prevent destruction
}

int good_return() {
    int x = 42;
    return x;  // ✅ OK: return by value (copy/move/elision happens)
}
```

The `std::move` in `bad_return_rvalue()` only casts `x` to an rvalue; it doesn't extend its lifetime or prevent its destruction. The function returns a reference to destroyed stack memory, leading to undefined behavior when the caller attempts to use it.

Lvalue and Rvalue References

Edge Cases

📌 5. Reference Binding and Overload Resolution

When multiple overloads exist, C++ uses complex rules to determine which function to call based on value categories.

```
void f(int& x)   { std::cout << "lvalue ref\n"; }
void f(int&& x)  { std::cout << "rvalue ref\n"; }
void f(const int& x) { std::cout << "const lvalue ref\n"; }

int main() {
    int a = 5;
    const int ca = 5;

    f(a);              // Calls f(int&) - exact match for lvalue
    f(5);              // Calls f(int&&) - exact match for rvalue
    f(std::move(a));   // Calls f(int&&) - std::move produces rvalue

    f(ca);             // Calls f(const int&) - const lvalue needs const ref

    // If f(int&&) didn't exist:
    // f(5) would call f(const int&) - rvalues can bind to const lvalue ref
}
```

The overload resolution prefers exact matches first. Lvalues prefer lvalue references, rvalues prefer rvalue references, but rvalues can fall back to const lvalue references if no rvalue reference overload exists.

Lvalue and Rvalue References

Code Examples

📌 1. Basic Value Category Identification

This example illustrates the fundamental distinction between lvalues and rvalues. Lvalues have persistent storage and can have their address taken, while rvalues are temporary and exist only within an expression. The compiler enforces these rules at compile time, preventing dangerous operations like taking the address of a temporary.

```
#include <iostream>
#include <type_traits>

int main() {
    int x = 10;
    int* ptr = &x;

    // Lvalue examples:
    int& lref = x;           // ✅ Variable x is an lvalue
    int& lref2 = *ptr;       // ✅ Dereferenced pointer is an lvalue
    int& lref3 = lref;       // ✅ lvalue reference itself is an lvalue

    // Rvalue examples:
    int&& rref1 = 42;        // ✅ Literal is an rvalue
    int&& rref2 = x + 5;     // ✅ Expression result is an rvalue
    int&& rref3 = std::move(x);  // ✅ std::move produces an rvalue

    // These would fail:
    // int& bad1 = 42;       // ❌ Cannot bind non-const lvalue ref to rvalue
    // int&& bad2 = x;       // ❌ Cannot bind rvalue ref to lvalue
    // int* bad3 = &42;      // ❌ Cannot take address of rvalue

    std::cout << "x = " << x << "\n";
    std::cout << "rref1 = " << rref1 << "\n";
}
```

Lvalue and Rvalue References

Code Examples

📌 2. Named Rvalue References Behavior

This demonstrates that naming an rvalue reference creates an lvalue. Once a temporary is bound to an rvalue reference parameter or variable, that reference itself becomes an lvalue that can be used multiple times. To pass it onward as an rvalue, you must explicitly use `std::move` to cast it back.

```
#include <iostream>
#include <utility>

void use_rvalue(int&& x) {
    std::cout << "Received rvalue: " << x << "\n";
}

void process(int&& param) {
    std::cout << "param has type int&& but is an lvalue expression\n";

    // param is an lvalue even though its type is int&&
    // use_rvalue(param);  // ❌ Error: param is lvalue

    use_rvalue(std::move(param));  // ✅ OK: cast back to rvalue

    // After this point, param is in valid but unspecified state
    std::cout << "param after move: " << param << "\n";  // ✅ Legal but value is unspecified
}

int main() {
    int&& rref = 100;

    // rref is an lvalue, even with type int&&
    int& lref = rref;  // ✅ OK: binding lvalue ref to lvalue

    // To pass rref as rvalue, must use std::move
    process(std::move(rref));
}
```

Lvalue and Rvalue References

Code Examples

📌 3. Overload Resolution with References

This example shows how overload resolution selects the appropriate function based on value categories. The compiler automatically chooses `process(int&)` for lvalues and `process(int&&)` for rvalues, enabling different behavior for temporary vs. persistent objects.

```
#include <iostream>

void process(int& x) {
    std::cout << "Lvalue reference: " << x << "\n";
    x = 999;  // Can modify
}

void process(int&& x) {
    std::cout << "Rvalue reference: " << x << "\n";
    x = 888;  // Can modify temporaries too
}

int getValue() { return 42; }

int main() {
    int a = 10;
    const int ca = 20;

    process(a);              // Calls process(int&)
    process(30);             // Calls process(int&&)
    process(getValue());     // Calls process(int&&)
    process(std::move(a));   // Calls process(int&&)

    std::cout << "a after move: " << a << "\n";  // Prints 888 (moved-from but valid)

    // process(ca);  // ❌ Error: no overload takes const int&
}
```

Lvalue and Rvalue References

Code Examples

📌 4. Const Lvalue References as Universal Binders

Before C++11, `const T&` was the standard way to accept both lvalues and rvalues without copying. It can bind to any value category, making it extremely flexible. However, it doesn't allow distinguishing between temporaries and non-temporaries, which is why rvalue references were needed for move semantics.

```
#include <iostream>

void accept_anything(const int& x) {
    std::cout << "Received: " << x << "\n";
    // x = 100;  // ❌ Cannot modify const reference
}

int main() {
    int a = 10;
    const int ca = 20;

    // const int& can bind to everything:
    accept_anything(a);              // ✅ Lvalue
    accept_anything(ca);             // ✅ Const lvalue
    accept_anything(30);             // ✅ Rvalue (temporary)
    accept_anything(a + 5);          // ✅ Rvalue (expression)
    accept_anything(std::move(a));   // ✅ Rvalue (moved)
}
```

Lvalue and Rvalue References

Code Examples

📌 5. Reference Collapsing Preview

This example previews universal references and reference collapsing, which will be covered in detail in the perfect forwarding topic. When `T&&` appears in a template, it's not just an rvalue reference—it's a forwarding reference that can bind to both lvalues and rvalues through reference collapsing rules.

```
#include <iostream>
#include <type_traits>

template<typename T>
void inspect(T&& param) {
    std::cout << "T = " << typeid(T).name() << "\n";

    if (std::is_lvalue_reference<T>::value) {
        std::cout << "T is lvalue reference\n";
    } else {
        std::cout << "T is not lvalue reference\n";
    }

    if (std::is_rvalue_reference<decltype(param)>::value) {
        std::cout << "param is rvalue reference\n";
    } else {
        std::cout << "param is not rvalue reference\n";
    }
}

int main() {
    int x = 10;

    inspect(x);              // T = int&, param becomes int& (lvalue ref)
    inspect(20);             // T = int, param becomes int&& (rvalue ref)
    inspect(std::move(x));   // T = int, param becomes int&& (rvalue ref)
}
```

Lvalue and Rvalue References

Code Examples

📌 6. Lifetime Extension with References

C++ extends the lifetime of temporaries when they're bound directly to references, whether const lvalue references or rvalue references. However, this only applies to direct binding—if a temporary is created inside a function and a reference to it is returned, the temporary is destroyed when the function returns, making the reference dangle.

```
#include <iostream>
#include <string>

const std::string& getConstRef() {
    return std::string("temporary");  // ❌ Dangerous: returning ref to temporary
}

std::string getValue() {
    return std::string("value");  // ✅ Safe: return by value
}

int main() {
    // Lifetime extension only works with direct binding:
    const std::string& ref1 = std::string("hello");  // ✅ Lifetime extended
    std::cout << "ref1: " << ref1 << "\n";

    // Also works with rvalue references:
    std::string&& ref2 = std::string("world");  // ✅ Lifetime extended
    std::cout << "ref2: " << ref2 << "\n";

    // Works with function returns:
    const std::string& ref3 = getValue();  // ✅ Lifetime extended
    std::cout << "ref3: " << ref3 << "\n";

    // Dangerous pattern - undefined behavior:
    // const std::string& bad = getConstRef();  // ❌ UB: temporary already destroyed
    // std::cout << bad << "\n";  // Accessing destroyed object
}
```

Lvalue and Rvalue References

Code Examples

📌 7. Type vs Value Category Distinction

This example explicitly demonstrates the critical but confusing distinction: `decltype(x)` tells you the type, while the expression `x` has a value category. A variable with rvalue reference type is still an lvalue expression because it has a name and storage.

```
#include <iostream>
#include <type_traits>

void analyze(int&& x) {
    std::cout << "Parameter type is rvalue reference: "
              << std::is_rvalue_reference<decltype(x)>::value << "\n";

    // But x as an expression is an lvalue:
    int& lref = x;  // ✅ OK: x is an lvalue
    std::cout << "Can bind x to lvalue reference\n";

    // To use x as rvalue, need std::move:
    // int&& rref = x;  // ❌ Error
    int&& rref = std::move(x);  // ✅ OK
    std::cout << "std::move(x) is rvalue\n";
}

int main() {
    int&& myref = 42;

    // myref has TYPE int&&
    std::cout << "myref type is rvalue reference: "
              << std::is_rvalue_reference<decltype(myref)>::value << "\n";

    // But myref as EXPRESSION is lvalue
    analyze(std::move(myref));  // Must use std::move to pass it
}
```

Lvalue and Rvalue References

Code Examples

📌 8. Multiple Reference Bindings

References can be chained, but the rules follow value categories. Lvalue references can freely refer to other lvalue references. Rvalue references, once named, become lvalues and require `std::move` to be passed to another rvalue reference.

```
#include <iostream>

int main() {
    int x = 100;

    // Multiple lvalue references to same object:
    int& ref1 = x;
    int& ref2 = ref1;  // ✅ OK: ref1 is also an lvalue
    int& ref3 = x;

    ref1 = 200;
    std::cout << "x: " << x << ", ref2: " << ref2 << ", ref3: " << ref3 << "\n";
    // All print 200

    // Rvalue references:
    int&& rref1 = 50;  // Binds to temporary

    // rref1 is lvalue, so:
    int& lref_to_rref = rref1;  // ✅ OK
    // int&& rref2 = rref1;  // ❌ Error: rref1 is lvalue
    int&& rref2 = std::move(rref1);  // ✅ OK

    rref2 = 75;
    std::cout << "rref1: " << rref1 << ", rref2: " << rref2 << "\n";
    // Both might print 75 (implementation-dependent after move)
}
```

What You Learned

Key Takeaways

- ✓ Understanding Value Categories
- ✓ Lvalue References vs Rvalue References
- ✓ The Critical Distinction
- ✓ Named Rvalue References Are Lvalues

→ Swipe to learn →